Q1: "When exactly does the Cleanup function run?"

- The cleanup function runs in two scenarios: Before the component unmounts (to clean up the final state).
- Before the effect re-runs due to a dependency change (to clean up the previous effect before starting the new one).

Q2: "Why is my useEffect entering an Infinite Loop?"

- This usually happens if you update a state inside useEffect that is also listed in the dependency array.
- Example: Setting count inside useEffect, but having [count] as a dependency. React sees count changed, re-runs the effect, which changes count again... loop.

Q3: "What happens if I put an Object or Array in the dependency array?"

- React compares dependencies using Referential Equality (===).
- If you pass an object options = { debug: true } into the dependency array, React sees it as a new object on every render (different memory address), causing the useEffect to run on every single render.
- Fix: Use useMemo for the object or destructure the primitive values inside the array.

Q4: "Why does useEffect run twice in development?"

- This is due to React.StrictMode. In development (only), React mounts, unmounts, and remounts components effectively to stress-test your cleanup functions. It ensures your logic is resilient. It does not happen in production.

Q5: "What is rendering?"

- Rendering in React is the process of taking component JSX and converting it into DOM elements.
- React renders the UI initially, and then re-renders components whenever state or props change.
- React uses the Virtual DOM to efficiently update only the parts of the UI that need to change.

Q6: "What is React.memo()?"

- `React.memo` is a higher-order component that prevents unnecessary re-renders of functional components by memoizing the rendered output. It re-renders a component only when its props change.
- We use it to improve performance, especially in large UI components, list rendering, or frequently re-rendering parent components.
- We use React.memo in child components that receive props but do not need to re-render frequently, such as cards, list items, dashboard widgets, or heavy components to avoid performance overhead.

Parent Comp

```
import { useState } from "react";
import Child from "./Child";
const Parent = () => {
  const [count, setCount] = useState(0);
  return (
    <>
      <h2>I am Parent Comp</h2>
      <h1>Count : {count}</h1>
      <button onClick={() => setCount(count + 1)}>Click</button>
      <Child value="Hello I am Child Props" />
    </>
  );
};
export default Parent;
```

Child Comp

```
import React from "react";
const Child = ({ value }) => {
  console.log(value);
  return (
    <>
      <h2>I am Child</h2>
    </>
  );
};
export default React.memo(Child);
```

Q7: "What is useCallback()?"

- useCallback() is a react hook used to optimized the performance of react application with the help of memorize function() that means this hook save and reuse a function until the dependency not change. This hook does not recreate the function on every re-render.

Parent Comp

```
import Child from "./Child";
import { useCallback, useState } from "react";
const Parent = () => {
  const [count, setCount] = useState(0);

  /* const eventHandler = () => {
    console.log("event trigger !!!");
  }; */

  const eventHandler = useCallback(() => {
    console.log("event trigger !!!");
  }, []);
  return (
    <>
      <h2>I am Parent </h2>
      <button onClick={() => setCount(count + 1)}>Count : {count}</button>
      <Child event={eventHandler} />
    </>
  );
};
export default Parent;
```

Child Comp

```
import React from "react";
const Child = ({ event }) => {
  console.log("Child Render");
  return (
    <>
      <h2>I am Child </h2>
      <button onClick={event}>click</button>
    </>
  );
```

```
};
export default React.memo(Child);
```

Q8: "What is useMemo()?"

- `useMemo` is a React Hook that memoizes the result of an expensive computation so it does not re-run on every render. It recomputes the value only when its dependencies change. It improves performance by preventing unnecessary recalculations.
- We use `useMemo` for expensive calculations, large list filtering, sorting, or any computation that does not need to execute on every render.

Q9: "What is Pagination?"

Pagination is the process of dividing a large dataset into smaller, manageable chunks (pages) to improve User Experience (UX) and Performance.

Q10: "What is Redux?"

"Redux is a Predictable State Container for JavaScript applications. It helps manage 'Global State' by storing the entire application's data in a single source of truth (The Store) and ensures that state changes only happen in a predictable way using pure functions called Reducers."

API Fetch

index.js

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { Provider } from "react-redux";
import store from "./redux/store";
import App from "./App";

const rootElement = document.getElementById("root");
const root = createRoot(rootElement);

root.render(
  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>
);
```

store.js

```
import { configureStore } from "@reduxjs/toolkit";
import todoReducer from "./slice";
const store = configureStore({
  reducer: {
    todo: todoReducer,
  },
});
export default store;
```

Slice.js

```
import { createAsyncThunk } from "@reduxjs/toolkit";
import { createSlice } from "@reduxjs/toolkit";
import axios from "axios";

const fetchTodoData = createAsyncThunk("fetchtododata", async () => {
  try {
    const response = await axios.get(
      "https://jsonplaceholder.typicode.com/todos"
    );
    console.log(response);
    return response.data;
  } catch (error) {
    console.log("failed", error);
  }
});

const todoSlicer = createSlice({
  name: "todo",
  initialState: {
    isLoading: false,
    data: [],
    error: null,
  },
  extraReducers: (builder) => {
```

```
    builder.addCase(fetchTodoData.pending, (state, action) => {
      state.isLoading = true;
      state.error = false;
    });

    builder.addCase(fetchTodoData.fulfilled, (state, action) => {
      (state.isLoading = false), (state.data = action.payload);
    });

    builder.addCase(fetchTodoData.rejected, (state, action) => {
      state.isLoading = false;
      state.error = action.payload;
    });
  },
});

export default todoSlicer.reducer;
export { fetchTodoData };
```

Todo.jsx

```
import { useEffect } from "react";
import { useSelector } from "react-redux";
import { useDispatch } from "react-redux";
import { fetchTodoData } from "../redux/slice";
const Tododata = () => {
  const dispatch = useDispatch();
  const { data, isLoading, error } = useSelector((state) => state.todo);

  useEffect(() => {
    dispatch(fetchTodoData());
  }, [dispatch]);

  if (isLoading) return <h2>Loading...</h2>;
  if (error) return <h2 style={{ color: "red" }}>{error}</h2>;
  return (
    <>
      <h2>Todo List</h2>
      {data.map((todo) => {
        return <p key={todo.id}>{todo.title}</p>;
      })}
    </>
  );
```

```
};
export default Tododata;
```

Counter App

Count_store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../redux/count_slice";
const countStore = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
export default countStore;
```

Count_slice.js

```
import { createSlice } from "@reduxjs/toolkit";

const countSlicer = createSlice({
  name: "counter",
  initialState: {
    value: 0,
  },
  reducers: {
    inc: (state) => {
      state.value += 1;
    },
    dec: (state) => {
      state.value -= 1;
    },
  },
});
export const { inc, dec } = countSlicer.actions;
export default countSlicer.reducer;
```

Counter.jsx

```
import { useReducer } from "react";
import { useDispatch, useSelector } from "react-redux";
import { inc, dec } from "../redux/count_slice";
const Counter = () => {
  const dispatch = useDispatch();
  const count = useSelector((state) => state.counter.value);
  return (
    <>
      <h1>Counter </h1>
      <button onClick={() => dispatch(inc())}>+</button>
      <button>{count}</button>
      <button onClick={() => dispatch(dec())}>-</button>
    </>
  );
};
export default Counter;
```

Q11: "What is slice in RTK?"

A Slice is a function provided by Redux Toolkit (createSlice) that accepts an initial state, a slice name, and an object of reducer functions. It automatically generates the Action Creators and Action Types for us, eliminating the boilerplate code we used to write in legacy Redux."

Q12: "What is custom hook in react?"

A custom hook in React is a reusable JavaScript function that uses React's built-in hooks (useState, useEffect, useMemo, etc.) to share logic between components without repeating code.

Q13: What is Event Delegation?

"Event Delegation is a technique where instead of adding event listeners to each child element, we add one event listener to their parent and use event bubbling to detect which child was clicked.
This improves performance, especially when elements are created dynamically.
In React, event delegation works naturally due to React's synthetic event system."

Q14: "How event delegation efficient?"

Because instead of attaching hundreds of listeners, we attach only one → less memory usage, improved performance.

Q15: "What is event bubbling?"

Event Bubbling is when an event starts from the target element and propagates upward to its ancestors (child → parent → document). It is default behavior.

parent, all the way to the root (document).

Child → Parent → Body → Document

Q16: "What is event capturing?"

Event Capturing is the opposite: event travels from parent → child. It is not default and must be enabled using addEventListener(event, handler, true).
Document → Body → Parent → Child

Capturing happens before bubbling.

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Playground</title>
    <style>
      #a{background-color : yellow; width : 300px; padding : 20px}
    </style>
  </head>
  <body >
    <div id='a'>
      <button id='btn'>Click Me</button>
    <div>

    <script src="src/script.js"></script>
  </body>
</html>
```

```javascript
const div = document.getElementById('a')
const bt  = document.getElementById('btn')
```

```
div.addEventListener('click',()=>{
  console.log('Parent Click')
},true) //true for event capturing without true event bubbling

bt.addEventListener('click',()=>{
  console.log('child Click')
})

//event bubbling - child click ----> parent click
//event capturing - parent click ---> child click
```

Q17: "What is code splitting?"

Code Splitting is a performance optimization technique where you break your JavaScript bundle into smaller chunks instead of sending the entire application to the user at once.

Q18: "What is lazy loading?"

"Instead of loading the entire React app in one big bundle, code splitting breaks it into smaller chunks. React loads only what the user needs at the moment, which makes the app load faster. When the user navigates to another route, the new chunk is downloaded on demand."

Lazy Loading = “Load components only when they are needed.”

Lazy loading is built on top of code splitting. It lets me delay loading a component until the user actually needs it—for example, loading the dashboard only when the user clicks on the dashboard route

Q19: "What is <Suspense> Comp?"

Suspense is a React component used to wrap lazy-loaded components. It shows a fallback UI while the component is being fetched. This prevents the UI from breaking or showing a blank screen during loading.

Q20: "What is Fallback?"

Fallback is the temporary UI displayed while a lazy-loaded component is being downloaded. It acts as a placeholder until the actual component finishes loading.

```
import axios from 'axios'
import React, { useEffect, useState } from 'react'
```

```
const Users = () => {
    const [users, setUsers] = useState([])
    const getData = async() =>{
        const res = await axios.get('https://dummyjson.com/users')
        const result = res.data.users
        //console.log(result)
        setUsers(result)
    }

    useEffect(()=>{
        getData()
    },[])
  return (
    <div>
        <h1>Users Details</h1>
        <div style={{marginTop : '25px'}}>
            {
                users.map((user)=>{
                    return(
                        <div  style={{backgroundColor  :  'white',  color  :  'black',
borderRadius : '20px', marginBottom : '10px'}} key={user.id}>
                            <div>Fullname - {user.firstName} {user.maidenName}
                            {user.lastName}</div>
                            <div>Age - {user.age}</div>
                            <div>DOB- {user.birthDate}</div>
                            <div>Blood Group- {user.bloodGroup}</div>
                        </div>
                    )
                })
            }
        </div>
    </div>
  )
}

export default Users
```

App.jsx

```
import React, { Suspense, useState } from 'react'
import './App.css'
//import Users from './components/Users'
const Users = React.lazy(()=>import('./components/Users'))
```

```
const App = () => {
  const [data,setData] = useState(false)
  const handleClick = () =>{
    setData(!data)
  }
  return (
    <div>
      <h1>Lazy Loading</h1>
      <button onClick={handleClick}>
        {
          data ? 'Hide Data' : 'Show Data'
        }
      </button>
      {
        data && (
        <Suspense fallback={<h3>Loading....</h3>}>
          <Users/>
        </Suspense>)
      }
      {/* {data && <Users/>} */}

    </div>
  )
}

export default App
```

Q21: What is a lighthouse?

Lighthouse is a built-in Chrome tool that audits a website's performance, accessibility, SEO, and best practices. I use it to analyze how fast my application loads, detect large JavaScript bundles, find accessibility issues, and improve overall user experience. It provides scores with suggestions that help optimize the app.

Q22. What is Debouncing?

"Debouncing is a performance optimization technique in JavaScript where we **delay the execution of a function until a certain amount of time has passed without the event being triggered again**. If the event continues to occur within that delay, the timer resets. This ensures the function executes **only once** after the user stops performing the action. "

The Technical Explanation: It works by delaying the execution of the function. Every time the event triggers, the previous timer is cleared (cancelled) and a new one is set. The function will only execute if the user stops triggering the event for the full duration of the delay.

The Analogy (Interview Gold): "It is like an elevator door. If someone walks in, the door stays open. It will only close once nobody has walked in for 5 seconds."

Primary Use Case: Search bar suggestions. You don't want to fire an API call for every letter typed (e.g., 'A', 'Ap', 'App', 'Appl', 'Apple'). You wait until the user stops typing for 300ms, then fire one request for "Apple".

```html
<body >
    <input type="text" placeholder="Search..." id='search'>
    <script src="src/script.js"></script>
  </body>
```

```javascript
let searchData = document.getElementById('search')
const debounceSearch = (fn,delay) =>{
  let timer;
  return function (...args){
    clearTimeout(timer)
    timer = setTimeout(()=>fn.apply(this,args),delay)
  }
}

const searchHadler = (e) =>{
  console.log('API Call',e.target.value)
}

const debounceHandler = debounceSearch(searchHadler,500)
//searchData.addEventListener('input',searchHadler)
searchData.addEventListener('input',debounceHandler)
```

For React Code...

Custom hook -

```
import { useEffect,useState } from "react"

const useDebounce = (value,delay) => {
  const [debounceValue, setdebounceValue] = useState(value)

  useEffect(()=>{
    let timer = setTimeout(()=>{
        setdebounceValue(value)

        return ()=>{
            clearTimeout(timer)
        }
    },delay)
  },[value,delay])

  return debounceValue
}

export default useDebounce
```

Comp -

```
import React, { useEffect, useState } from "react";
import useDebounce from "../hooks/useDebounce";

const Debounce = () => {
  const [text, settext] = useState("");
  const debounceSearch = useDebounce(text, 1000);

  useEffect(() => {
    console.log('without debounce - ',text)
  },[text]);
```

```
  useEffect(()=>{
    console.log('with debounce -',debounceSearch)
  },[debounceSearch])
  return (
    <div>
      <div>
        <input
          type="text"
          placeholder="Enter the Value"
          value={text}
          onChange={(e) => settext(e.target.value)}
        />
      </div>
      <div>Current Data - {text}</div>
    </div>
  );
};

export default Debounce;
```

Q23. What is Throttling ?

Throttling is a performance optimization technique in JavaScript where we **limit how often a function can execute within a specified time interval**, even if an event keeps firing continuously. Once the function runs, it will not run again until the defined time period has passed.

The Technical Explanation: It ensures a function executes at a steady pace. When the event triggers, if the function has run recently (within the time limit), the new call is ignored. Once the time limit expires, the function is allowed to run again.

Primary Use Case: Infinite scrolling or Window resizing. As a user scrolls, thousands of events fire. You don't want to calculate page height 1000 times a second; checking it once every 100ms is sufficient to load new content.

```
function throttle(fn, delay) {
  let last = 0;
```

```
  return function (...args) {
    const now = Date.now();
    if (now - last >= delay) {
      last = now;
      fn.apply(this, args);
    }
  };
}

function handleScroll() {
  console.log("Scroll Event:", new Date().toLocaleTimeString());
}
//window.addEventListener("scroll", handleScroll, 1000);
window.addEventListener("scroll", throttle(handleScroll, 1000));
```

For React -

Custom hook -

```
import React, {useEffect, useRef, useState } from 'react'

const useThrottle = (value,limit) => {
    const [throttleValue,setThrottleValue] = useState(value)
    //to get current time we use useRef ()
    const lastTime = useRef(Date.now())
    //console.log(lastTime);

    useEffect(()=>{
        let handler = setTimeout(()=>{
            if(Date.now() - lastTime.current >= limit){
                setThrottleValue(value)
                lastTime.current = Date.now() //to reset
            }
        }, limit - (Date.now() - lastTime.current))
        return ()=>{
            clearTimeout(handler)
        }
```

```
    },[value,limit])

  return throttleValue
}

export default useThrottle
```

Comp -

```
import React, { useState } from 'react'
import useThrottle from '../hooks/useThrottle'



const Throttledemo = () => {
    const [text,setText] = useState("")
    const throttledValue = useThrottle(text,2000)
  return (
    <div>
        <h2>Throttle Comp</h2>
        <div>
            <input
              type='text'
              placeholder='Enter Value'
              value={text}
              onChange={(e)=>setText(e.target.value)}/>
        </div>
        <p>
          Live : {text}
        </p>
        <p>
          Throttle : {throttledValue}
        </p>
    </div>
  )
}
```

```
export default Throttledemo
```

Q24. What is useEffect -

"useEffect is a hook used to synchronize a component with an external system. External systems include things not controlled by React, such as the Browser DOM, Network requests (APIs), Timers, or Subscriptions. It handles side effects in functional components."

Q25. What is CORS?

CORS stands for Cross-Origin Resource Sharing. It is a browser-enforced security feature that restricts web pages from making requests a different origin for security reasons. When a frontend and backend run on different domains, the browser sends a preflight OPTIONS request to check permissions. The backend must respond with headers such as `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, and `Access-Control-Allow-Headers`. If these are valid, the browser allows the actual request. CORS protects APIs from unauthorized cross-site access while still enabling controlled communication across origins.

Q26. What is virtualization in react?

Virtualization, or Windowing, is a critical performance technique we use in React to manage and efficiently render extremely long lists—typically those with thousands of items—that would otherwise crash or heavily degrade the user experience.

Q27. What is clamp () in CSS?

clamp() is a CSS function that allows you to set a responsive value with a minimum, preferred, and maximum limit.

clamp(minimum, preferred, maximum)

Q28. What is ARIA?

ARIA (Accessible Rich Internet Applications) is a set of HTML attributes that help make web applications accessible to screen readers and assistive technologies when native HTML alone is not enough.

👉 ARIA provides extra information to users who cannot see the UI.

Q29. What is Closure?

A Closure is created when a function is defined inside another function, allowing the inner function to access variables from the outer function's scope—even after the outer function has finished executing.

1. **Data Privacy / Encapsulation**: Since JavaScript didn't have private variables (before classes #), closures were the primary way to emulate private methods (The Module Pattern).
2. **State Retention:** Useful for things like maintaining the state of a counter, or an iterator, without polluting the global scope.
3. **Functional Programming:** Closures are the backbone of techniques like Currying (transforming a function with multiple arguments into a sequence of nesting functions) and Memoization (caching results).

```
function createBankScote(initialBalance) {
  // This variable is "private" to this scope
  let _balance = initialBalance;
  return {
    deposit: function(amount) {
      _balance += amount;
      return _balance;
    },
    withdraw: function(amount) {
      if (amount > _balance) {
        return "Insufficient funds";
      }
      _balance -= amount;
      return _balance;
    },
    getBalance: function() {
      return _balance;
    }
  };
}
const myAccount = createBankScote(100);
console.log(myAccount.getBalance()); // 100
myAccount.deposit(50);
console.log(myAccount.getBalance()); // 150
```

```
// console.log(myAccount._balance); // undefined (Cannot access directly!)
```

Q30. What is Caching?

Caching is the process of storing copies of data or files in a temporary storage location (the cache) so that future requests for that data can be served faster than retrieving it from the primary source (like a database or API)

Q31. What is Memoization?

Memoization is an optimization technique used to speed up computer programs by storing the results of expensive function calls and returning the cached result when the same inputs occur again.

```
function memoize () {
  let cache = {}
  return function (n){
    if(n in cache){
      return `${cache[n]} - from cache`
    }else{
      cache[n] = n*n
      return cache[n]
    }
  }
}
let mymemo = memoize()
console.log(mymemo(5))
console.log(mymemo(5))
console.log(mymemo(6))
```

Q32. What is Function Currying ?

Function currying is a function programming technique in which we can transform a function into a sequence of function, each function taking a single argument.

OR

Currying is a technique where a function that normally takes multiple arguments is converted into a series of nested functions, each accepting one argument.

I've used currying in frontend to create reusable event handlers, reusable filter logic, and configurable utility functions.

However, I use it carefully because excessive currying can hurt readability and performance.

Older way -

```
function multiply(a) {
  return function(b) {
    return function(c) {
      return a * b * c;
    }
  }
}
console.log(multiply(2)(3)(4));
```

Arrow Function Way -

```
const multiply = a => b => c => a * b * c;
```

Q33. What is this keyword ?

this refers to the object that is currently executing the function. Its value depends on how the function is called — global, object method, event handler, strict mode, and when using call/apply/bind.

Q34. What is difference between arrow function and normal function ?

A normal function has its own this, arguments, and can be used as constructor.

Arrow function does not have its own this; it captures this lexically from parent scope.

Arrow functions are best for callbacks and functional programming, while normal functions are better for object methods, constructors, and places requiring dynamic this.

OR

1. **`this` Binding (The Key Difference) -**
   - In a Normal Function, the value of `this` is dynamic - it depends on how the function is called. If called as an object method, `this` is the object. If passed as a callback, `this` is often lost.
   - In contrast, an Arrow Function has a lexical `this`. It doesn't have its own context; it captures `this` from the parent scope where it was defined. This makes arrow functions ideal for React callbacks, as we don't need to manually `bind` them.
2. **Constructors** -
   - Normal functions are constructible—I can use the new keyword to create instances.

- Arrow functions are not; they lack a `prototype` property and will throw an error if used with `new`.

3. **Arguments Object** -

- Normal functions have access to the `arguments` keyword (an array-like object).
- Arrow functions do not have this; if I need to access variable arguments in an arrow function, I use the Rest Operator (`...args`).

```
const Person = {
  name : 'Apurva',
  greetN : function (){
    console.log(this.name)
  },
  greetA : () =>{
    console.log(this.name)
  }
}
Person.greetN()           //Apurva
Person.greetA()           //undefined
function User  (name,age)  {
  this.name = name
  this.age = age
}


const user1 = new User('Mayur',32)
console.log(user1)                      // User {name: "Mayur", age: 32}

//if I use arrow function here then it now work with new keyword
```

Q35. What is call(), apply() and bind() ?

1. `call()`

- What it does: Invokes the function immediately.
- Arguments: You pass arguments one by one (comma-separated).
- Mnemonic: Call takes Commas.

2. `apply()`

- What it does: Invokes the function immediately.
- Arguments: You pass arguments as a single Array (or array-like object).
- Mnemonic: Apply takes an Array.

3. `bind()`

- What it does: Does NOT run the function immediately. It returns a new function copy with this permanently "bound" to the object you passed.
- Arguments: Same as call (one by one), but these are usually "preset" arguments.
- Use Case: Essential in React class components or event listeners where you want to run the function later but keep the context safe.

Interview-Ready Answer (What to say)

"All three are used to manipulate the `this` context of a function.

- call and apply invoke the function immediately. The difference is that call accepts arguments individually, whereas apply accepts them as an array.
- bind is different because it doesn't run the function right away. Instead, it returns a new function instance with the this context permanently bound to the object we passed in. This is useful for event handlers or partial application."

```
const user1 = {
  name : 'Mayur',
  company : 'chat360'
}
const user2 = {
  name : 'Apurva',
  company : 'chat360'
}
function intro (salary) {
  console.log(`I am ${this.name}, My current company is ${this.company}, current ctc is
${salary} LPA`)
}

//Call() method
intro.call(user1,10)
```

```
intro.call(user2,8)

//Apply() Method
intro.apply(user1,[10])
intro.apply(user2,[8])


//Bind() method
const u1 = intro.bind(user1,10)
u1()
const u2 = intro.bind(user2,8)
u2()
```

output

```
I am Mayur, My current company is chat360, current ctc is 10 LPA
I am Apurva, My current company is chat360, current ctc is 8 LPA
I am Mayur, My current company is chat360, current ctc is 10 LPA
I am Apurva, My current company is chat360, current ctc is 8 LPA
I am Mayur, My current company is chat360, current ctc is 10 LPA
I am Apurva, My current company is chat360, current ctc is 8 LPA
```

Q36. What is Prototype in JS?

A prototype is an object that JavaScript uses as a fallback source of properties and methods. Every JavaScript object has an internal link to another object called its `prototype`.

So if a property or method is not found on the object, JavaScript automatically looks for it in its prototype.

OR

In JavaScript, every object has an internal property called `[[Prototype]]` which refers to another object. This prototype acts as a fallback, so if a property or method is not found on the object, JavaScript looks it up in the prototype chain.

Q37. What is Prototype Inheritance?

Prototype inheritance means that objects in JavaScript can inherit properties and methods from another object through the prototype chain.

OR

Prototype inheritance means objects can inherit properties and methods from other objects through this prototype chain. Functions also have a prototype object, and when we create objects

using new, those objects inherit from the function's prototype. This enables reusability, memory optimization, and object-based inheritance in JavaScript.

Q38. What is Promise?

In JavaScript, a Promise is an object that represents the eventual completion or failure of an asynchronous operation. It acts as a placeholder for a value that is not available yet but will be in the future.

A promise exist in one of three state pending, fulfilled, reject

To handle Promises, I use instance methods like .then() for success, .catch() for errors, and .finally() for cleanup logic that runs regardless of the outcome.

In real-world applications, I often handle multiple asynchronous operations simultaneously using Static Methods. The four most important ones are:

- **`Promise.all()`:**
  - **Behavior:** Waits for **all** promises to fulfill.
  - **Success:** Returns an array of all results `[r1, r2, r3]`.
  - **Failure: "Fail-Fast"**. If *any* promise rejects, the whole thing rejects immediately with that error. The other successes are ignored.
  - **Use Case:** You need 3 API calls to render a page, and if one fails, the page is broken.
- **`Promise.allSettled()`:**
  - **Behavior:** Waits for **all** promises to finish, regardless of whether they succeeded or failed.
  - **Result:** Returns an array of objects describing the outcome: `{ status: 'fulfilled', value: ... }` or `{ status: 'rejected', reason: ... }`.
  - **Use Case:** Fetching data for independent dashboard widgets. If one fails, you still want to show the others.
- **`Promise.race()`:**
  - This returns the result of the **first** promise to settle (success or failure). I often use this for implementing timeouts.
- **`Promise.any()`:**

- This is similar to `race` but it waits for the first **successful** resolution, ignoring failures unless all of them fail."

Q38. What is async and await?

Async/Await is modern syntax introduced in ES2017 to handle asynchronous operations more cleanly. It is essentially 'syntactic sugar' built on top of Promises.

- The async keyword: Declares that a function handles asynchronous operations and guarantees that it returns a Promise.
- The await keyword: Pauses the execution of the function until the Promise resolves, effectively unwrapping the result."

Q38. What is event loop?

The Event Loop is a mechanism that allows JavaScript to perform non-blocking asynchronous operations, despite being single-threaded, by offloading operations to the browser and handling their results when the Call Stack is empty.

Proper Interview Answer (Script)

"JavaScript is single-threaded, meaning it has one Call Stack and can do one thing at a time. However, to handle asynchronous operations without blocking the UI, it uses a mechanism called the Event Loop."

"Here is how it works:

1. Synchronous code runs immediately in the Call Stack.
2. Asynchronous tasks (like `setTimeout` or `fetch`) are offloaded to the browser's Web APIs.
3. Once those tasks complete, their callbacks are moved to a Queue.
4. The Event Loop has one simple job: it continuously monitors the Call Stack and the Queue. If the Stack is empty, it pushes the first task from the Queue into the Stack to be executed."

"Crucially, there are two types of queues:

- The Microtask Queue (for Promises) has higher priority.
- The Macrotask Queue (for `setTimeout`) has lower priority. The Event Loop will always process all Microtasks before moving on to the next Macrotask."

Q39. What is Method Chaining?

Method chaining is a technique where multiple methods are called on the same object in a single statement, one after another.

```
class Calculator {
  constructor(){
    this.value = 0;
  }

  add(num){
    this.value += num;
    return this;
  }

  subtract(num){
    this.value -= num;
    return this;
  }

  multiply(num){
    this.value *= num;
    return this;
  }
}

const calc = new Calculator();
calc.add(10).subtract(3).multiply(2);

console.log(calc.value); // 14
```

Promise Based Coding

```
//fetch API
const fetchData = (url) =>{
    return new Promise((resolve,reject)=>{
        fetch(url)
```

```
        .then((res)=>{
            if(!res.ok){
                reject(`Failed to fetch Data, ${res.status}`)
            }
            return res.json()
        })
        .then((data)=>resolve(data))
        .catch((error)=>reject(error))
    })
}
fetchData('https://jsonplaceholder.typicode.com/users')
.then((data)=>console.log(data))
.catch((error)=>console.log(error))
```

PERSONAL INTRODUCTION SCRIPT

“Hi, I’m Mayur, a React and Next.js developer with 4 years of experience building scalable and high-performance web applications.

My expertise includes React.js, Next.js, Context API, REST API integration, performance optimization, responsive UI design, and working with Node.js and MongoDB on the backend.

I’ve worked on two major projects recently:

Asset Tracker, where I implemented role-based UI, asset allocation workflows, and optimized React performance; and ChangeExpert, an online learning platform built using Next.js with SSR, JWT authentication, enrollment system, and strong SEO optimization.

Across all my projects, I focus on clean architecture, reusable UI components, optimized rendering, and delivering smooth user experiences.

I collaborate closely with backend teams, write clean, maintainable code, and I’m comfortable handling end-to-end development from UI to API integration.

I’m confident that my technical experience and hands-on approach will allow me to contribute effectively to real-world client projects.

CHANGEEXPERT

“ChangeExpert is an online learning platform where students can browse courses, view details, enroll, and access learning content.

I built the frontend using Next.js for SEO-friendly SSR and fast performance.

Many core pages like Course Details, Instructor Profiles, and Category Pages are server-rendered using getServerSideProps, ensuring the content loads instantly and is indexable by search engines.

Students can log in, enroll in courses, watch videos, and track their progress. Authentication is JWT-based, stored in HTTP-only cookies for better security.

I used dynamic imports for video components to improve load speed and Next.js Image optimization to reduce bundle size.

On the backend, Node.js and Express handle course, user, and enrollment APIs. MongoDB stores all course content, user details, and enrolled courses.

I implemented pagination for large course lists, caching for repeated API calls, and used Tailwind CSS for fast UI development.

Overall, I handled the entire Next.js part, SEO implementation, API integration, authentication, and performance optimization."

ASSET TRACKER

Short -

The Asset Tracker is an internal company tool used for managing employee assets such as laptops, monitors, keyboards, and access cards.

The system provides role-based access: Admins can allocate assets, update status, add new hardware, and manage employee records. Employees can view their assigned assets and raise issues.

I built the frontend using React and Context API, while the backend is built with Node.js, Express, and MongoDB.

Long -

Asset Tracker is an internal company tool to manage physical assets like laptops, monitors, keyboards, and ID cards.

I built the frontend in React and used Context API for managing authentication, user roles, and asset state.

Admins can add assets, allocate them to employees, update their status, and track the full lifecycle. Employees can view their assigned assets and raise issues.

I created reusable components for lists, filters, status badges, and modal forms. API integration is done using Axios, and all sensitive routes are protected using JWT authentication.

The backend is built with Node.js and Express.js.
For every request, JWT middleware checks the user role.
MongoDB stores user data and asset information.
I designed the database schema to keep asset status, serial number, and assigned user references.
I also implemented performance optimization using `React.memo`, `useCallback`, and lazy loading for admin screens.
To handle large data, I implemented server-side filtering and pagination.
Overall, I worked end-to-end on the frontend, collaborated with backend teams for API contracts, and ensured the UI remained responsive and clean.”